AI-POWERED BUG TRACKING SYSTEM

Software Specification

Employee and engineering issue reporting, triage and resolution tracking

Version 1.0 | 29 October 2026 | Draft for stakeholder review

Purpose and intended outcome

Build a web application that allows team members to report software bugs, track progress, comment on issues and receive updates. Developers and triage leads use the same application to assign work, manage priority and status, record fixes, and verify outcomes. AI assistance may suggest categories, similar bugs, summaries and draft tests, but people remain responsible for approving decisions.

Source requirement

“Teams need a consistent way to report, assign, track and verify software bugs, with AI helping reduce repetitive triage work.” This specification follows the structure and level of detail of the user-provided IT_Helpdesk_Specification.pdf. Requirements marked S are core product needs; requirements marked P are proposed elaborations for an implementable and testable draft.

How to read this specification

‘Shall’ describes required behavior in this draft. Proposed rules are a baseline for stakeholder review, not confirmed business facts. Requirement IDs connect implementation tasks, acceptance criteria and tests. AI assistance is included in this product scope; generated suggestions are never treated as confirmed facts without review.

Document map

Section	Coverage
1–2	Scope, assumptions, actors and access control
3–5	Functional requirements, bug lifecycle and web screens
6–8	Data model, REST API, errors, AI behavior and notifications
9–10	Quality requirements, acceptance criteria and test cases
11	Delivery sequence, change control and open decisions

1. Scope and baseline assumptions

Included in the initial release

• Users create and view permitted bug reports, add comments and read updates.

• Developers and triage leads view project bugs, assign work, set severity and priority, change status and record resolution details.

• The system keeps persistent issue history and in-app notifications.

• AI assists with category/priority suggestions, similar-bug discovery, report summaries and draft test cases.

• All user-facing actions are enforced by server-side permissions. AI suggestions require human review.

ID	Basis	Requirement
S01	S	Users shall be able to report software bugs.
S02	S	Users shall be able to track bug status and receive updates.
S03	S	The system shall support creating, viewing, updating and commenting on bugs.
S04	S	Required fields shall be validated and permissions enforced.
S05	S	The system shall expose a web form and REST API.
S06	S	A newly created bug shall start with status New.
S07	S	Verification shall include valid, invalid, unauthorized and AI-failure scenarios.
S08	S	AI suggestions shall be reviewable and shall not silently change confirmed fields.

Proposed assumptions

• A01 — One organization with projects in the first release. Roles: Reporter, Developer, Triage Lead and Administrator. User identity is provided by an external identity system.

• A02 — Statuses: New, Triaged, In Progress, Fixed, Ready for Verification, Closed and Reopened. Severity: Critical, High, Medium, Low. Priority: P0, P1, P2, P3.

• A03 — Updates mean persistent in-app notifications and issue activity history. Email, SMS, Slack and Teams notifications are excluded initially.

• A04 — AI uses a configured model/service behind a replaceable integration boundary. If the AI service fails, manual bug tracking continues.

• A05 — Users can report bugs; developers and triage leads can manage assigned/project bugs according to role and project access.

Excluded from this release

• Automatic code changes, autonomous merges or production deployments.

• Automatic closure or merging of duplicate bugs based only on AI.

• Full replacement for source control, CI/CD or test-management tools.

• Public bug submission, billing, multi-organization tenancy and advanced SLA escalation unless separately approved.

2. Actors and access control

Capability	Reporter	Developer	Triage Lead	Administrator
Create bug	Yes	Yes	Yes	Yes
Read bugs	Own/project permitted	Project permitted	All assigned projects	Configured scope
Comment	Permitted bugs	Permitted bugs	Permitted bugs	Permitted bugs
Edit title/description after submit	No	No by default	Yes with history	Yes with history
Change severity/priority	No	Limited by project policy	Yes	Yes
Assign/reassign	No	If permitted by project	Yes	Yes
Change status	No	Assigned bugs	Project bugs	Project bugs
Manage users/roles	No	No	No	Yes
Approve AI suggestions	May provide feedback	Yes	Yes	Yes
Delete bug/history	No	No	No	No

Authorization rules

• P01 — Every page and API operation containing bug, comment, attachment or notification data requires an authenticated active account.

• P02 — The server derives user identity and role from validated authentication context; client-supplied role or owner values never grant access.

• P03 — A user can only access projects and bugs permitted by their membership and role. Search results, pagination totals, comments and attachments obey the same rule.

• P04 — Unauthenticated requests return 401; authenticated but forbidden actions return 403. Missing and inaccessible bugs return the same 404 response.

• P05 — AI context must contain only bugs and project data the requesting user is allowed to access.

Primary user journeys

Reporter: sign in → create bug → receive reference and New status → open My/Project Bugs → follow comments and status → review fix → provide verification feedback.

Developer: sign in → open assigned bugs → inspect reproduction steps and evidence → update progress → add fix notes → mark Fixed → submit for verification.

Triage Lead: review new reports → inspect AI suggestions → confirm category/severity/priority → check possible duplicates → assign owner → monitor resolution.

3. Functional requirements

FR01 — Create bug

On valid submission, create one bug with unique immutable ID/reference, authenticated reporter, status New and server-generated UTC timestamps.

FR02 — Validate fields

Title and description are required. Invalid, blank, out-of-range or unknown fields fail without partial bug, history or notification records.

FR03 — View and search

List and detail views show reference, title, description, status, severity, priority, reporter, assignee, labels, timestamps and activity. Support search and filters for status, priority, severity, assignee, label and project.

FR04 — Project scope

Users see only bugs from projects they are authorized to access. Triage leads can view project-wide bugs within their scope.

FR05 — Update bug

Authorized users can update permitted fields. Changes record old/new values and use version checking to prevent overwriting newer changes.

FR06 — Assignment

Authorized triage leads assign or reassign a bug to an active project member. Assignment changes are recorded in history.

FR07 — Lifecycle

Status changes follow Section 4. A bug cannot be Closed without verification evidence or an approved closure reason.

FR08 — Comments

Authorized users can append plain-text comments. Comments are timestamped and immutable in the initial release.

FR09 — Activity history

Creation, field changes, status transitions, comments and AI suggestion decisions produce activity events with actor and server time.

FR10 — AI triage

AI may suggest category, severity/priority and a concise summary. Suggestions include a rationale or supporting report details and require explicit human confirmation before saving.

FR11 — Similar-bug suggestions

AI may suggest potentially similar bugs visible to the current user. It shall not automatically merge, close or mark bugs as duplicates.

FR12 — Test-case drafts

AI can draft reproduction or verification test cases from the report. Drafts remain unapproved until a user reviews and accepts them.

FR13 — AI uncertainty

If information is missing, AI should flag the gap or ask for clarification rather than inventing steps, logs or root causes.

FR14 — Notifications

Create in-app notifications for assignment, comments, status changes and verification requests, based on the rules in Section 8.

FR15 — Dashboard

Show counts by status, severity, priority, assignee and project, plus time to triage and resolution when data exists.

FR16 — Attachments

If enabled for release, validate file type and size, restrict access, and prevent uploaded content from executing. If not implemented, do not show a nonfunctional upload control.

FR17 — AI failure fallback

AI timeout or provider error must not block manual bug creation, editing, assignment or comments.

FR18 — Audit and feedback

Record whether each AI suggestion was accepted, edited or rejected. Do not treat feedback as proof that the suggestion was correct.

4. Bug lifecycle and business rules

From	To	Conditions
New	Triaged	Triage lead confirms classification and minimum details.
Triaged	In Progress	Active developer assignee required.
In Progress	Fixed	Fix summary or change reference required.
Fixed	Ready for Verification	Developer submits fix for verification.
Ready for Verification	Closed	Authorized verifier records successful verification evidence.
Ready for Verification	Reopened	Verification fails; reason required.
Closed	Reopened	Authorized triage lead reopens with reason if issue recurs.

• P06 — All other status transitions are rejected with 409 INVALID_TRANSITION; failed transitions leave the bug unchanged.

• P07 — In Progress bugs must have an active assignee. Reassignment records previous and new owner.

• P08 — Closing requires a meaningful verification result and verifier identity. A failed verification must include a reason.

• P09 — Reopening preserves prior fix and verification history; it does not erase earlier events.

• P10 — Bug title, description and comments are stored as text and safely escaped on display.

• P11 — A multi-field update, version increment, activity event and notification records commit atomically.

• P12 — Retrying a create/comment request with the same idempotency key returns the original result rather than creating duplicates.

Example scenario

A reporter submits “Application crashes when exporting CSV.” The bug starts New. A triage lead reviews an AI-suggested category and similar reports, confirms priority and assigns a developer. The developer reproduces the issue, implements a fix and marks it Fixed. A verifier tests the fix and either closes the bug with evidence or reopens it with a reason.

5. Web interface specification

Screen	Required content and behavior
Sign in	Use configured identity provider; return to intended authorized page after sign-in.
New Bug	Required title and description, reproduction steps, expected/actual result, environment, severity/priority and labels as agreed; field-level errors.
My/Project Bugs	Permitted bugs, search, filters, pagination and create action. Default sort by most recently updated.
All Bugs / Triage Queue	Triage Lead view of authorized project bugs; untriaged and unassigned filters.
Bug Details	Reference, description, environment, status, severity, priority, reporter, assignee, timestamps, activity, comments and fix/verification information.
AI Assistance panel	Clearly label suggestions; show rationale/uncertainty; allow accept, edit, reject; no silent writes.
Test Drafts	Show generated test cases as drafts with approve/edit/reject controls.
Notifications	Newest first, unread indicator, mark-as-read action; links to accessible bugs only.
Dashboard	Counts and simple metrics with filters; omit unsupported metrics rather than showing fake data.

Form and error behavior

• UI01 — Required fields are visibly labelled; whitespace-only text fails validation. Client-side validation mirrors but does not replace server validation.

• UI02 — Do not show success until the server confirms persistence. Retain form input on validation/network failure.

• UI03 — On version conflict, explain that the bug changed, refresh current data and preserve unsent user input; never silently overwrite.

• UI04 — Empty lists say “No bugs found” and allow clearing filters. Loading indicators and retry actions are provided.

• UI05 — 401 returns user to sign-in; 403 explains the action is unavailable; 404 says “Bug not found or unavailable.”

• UI06 — Controls support keyboard use, visible focus and accessible labels. Status is shown as text, not color alone.

6. Data model and validation

Field	Type / rule
id / reference	Server UUID and unique readable reference such as BUG-000123; never reused.
title	Required string, proposed 5–150 characters; immutable after creation unless triage policy permits an audited edit.
description	Required string, proposed 10–5,000 characters.
reproduction_steps	Plain text or ordered list; recommended, may be completed during triage.
expected_result / actual_result	Text; capture when applicable.
environment	OS, browser, app version, build and relevant runtime details.
severity	Critical / High / Medium / Low.
priority	P0 / P1 / P2 / P3; distinct from severity.
status	New, Triaged, In Progress, Fixed, Ready for Verification, Closed, Reopened.
reporter_id / assignee_id	User references; reporter required; assignee may be null where workflow permits.
project_id / labels	Project foreign key and searchable labels.
created_at / updated_at	Server-generated ISO 8601 UTC timestamps.
fix_summary / verification_result	Text required at relevant lifecycle transitions.
version	Positive integer, starts at 1; increments on committed changes.
AI suggestion record	Type, proposed value, rationale, model/version, timestamp and user decision.
Comment / ActivityEvent	Related records with author/actor, timestamp and immutable history.
Notification / IdempotencyRecord	Recipient/event, read state and safe retry record.

Persistence rules

• Enforce unique bug IDs/references, valid enums, required fields and foreign keys at persistence boundaries.

• Index project_id, reporter_id, assignee_id, status, priority and updated_at for scoped lists.

• Do not store secrets in bug text or AI prompts by design; provide guidance to redact credentials and tokens.

• Use UTC timestamps and preserve historical identity references for audit events.

7. REST API contract

Base path: /api/v1. Requests and responses use application/json. Protected endpoints require authentication. Lists return {items, page, page_size, total}. Default page_size is 20; proposed maximum is 100.

Method and path	Contract
POST /bugs	Create for current user. Body: title, description, reproduction_steps, expected_result, actual_result, environment, severity, priority, project_id, labels. Idempotency-Key required. Returns 201.
GET /bugs	List permitted bugs; supports status, severity, priority, assignee, project, label, q and pagination filters.
GET /bugs/{id}	Return permitted bug or 404; include ETag with current version.
PATCH /bugs/{id}	Update allowed fields/status; If-Match required; enforce role and lifecycle rules.
GET /bugs/{id}/activity	Return chronological comments and activity events.
POST /bugs/{id}/comments	Body: {body}; Idempotency-Key required; returns 201.
POST /bugs/{id}/ai-triage	Return labeled category/priority/summary suggestions; no automatic saved-field changes.
POST /bugs/{id}/similar	Return permitted possible duplicates with similarity rationale; no automatic merge.
POST /bugs/{id}/test-drafts	Return draft test cases for review.
PATCH /ai-suggestions/{id}	Record accept/edit/reject decision for an authorized suggestion.
GET /notifications	Current user's notifications, optionally unread-only.
PATCH /notifications/{id}	Mark current user's notification read.
GET /projects/{id}/members	Return authorized project members for assignment controls.

Error contract

Errors use {"error":{"code":"...","message":"...","fields":{},"request_id":"..."}}. Unknown fields are rejected.

HTTP	Meaning
400	MALFORMED_JSON — body cannot be parsed.
401 / 403	UNAUTHENTICATED / FORBIDDEN.
404	NOT_FOUND — resource missing or inaccessible.
409	INVALID_TRANSITION, DUPLICATE_CONFLICT or IDEMPOTENCY_CONFLICT.
412 / 428	VERSION_CONFLICT / PRECONDITION_REQUIRED.
422	VALIDATION_ERROR — invalid field, enum, filter or missing required input.
429	RATE_LIMITED — too many requests, including AI requests.
500 / 503	INTERNAL_ERROR / UNAVAILABLE — no internal details exposed.

8. AI behavior, notifications and reliability

AI guardrails

• AI outputs are suggestions, not authoritative facts. Human confirmation is required for saved category/priority changes, duplicate linking and official test cases.

• Use only data the current user is allowed to access. Do not reveal private or cross-project bugs as context.

• Validate AI output against a structured schema before displaying or storing it.

• Treat bug report text, comments and logs as untrusted input; they must not override system instructions or authorize actions.

• Do not allow AI to execute code, modify repositories, close bugs or merge duplicates in this release.

• On timeout or provider failure, show a clear retryable message and keep core tracking features available.

• Store model/provider version and user decision where needed for audit; avoid retaining unnecessary sensitive prompt content.

Notification rules

Committed event	Recipients
Bug created	Reporter receives confirmation.
Assignment changed	Reporter, previous assignee and new assignee, excluding actor.
Status or priority changed	Reporter and current assignee, excluding actor.
Comment added	Reporter and current assignee, excluding author.
AI suggestion decision	Relevant bug activity is updated; notify only if required by project settings.

• Create at most one notification per recipient per event; ignore null or inactive recipients.

• Notification summaries include bug reference and event type, not full descriptions, comments or secrets.

• Mark-as-read is explicit; viewing a list does not mark items read.

• Bug mutation, history event and notification writes should be atomic in the initial single-database design.

• AI endpoints should have configurable timeouts and rate limits. AI service failure must not roll back an otherwise valid manual bug operation.

9. Quality and operational requirements

ID	Requirement and verification
NF01 Security	Use HTTPS outside local development; enforce auth, authorization and server-side validation on every request; use parameterized queries and escape rendered text.
NF02 Privacy	Minimize data sent to AI; redact credentials/secrets; document provider retention and access settings before production.
NF03 Performance	Proposed target: common list/detail p95 ≤1 second and writes p95 ≤2 seconds in a defined test environment; measure AI latency separately.
NF04 Usability	At 375px and 1440px viewport widths, primary actions remain usable without page-level horizontal scrolling; test keyboard-only flows.

ID	Requirement and verification
NF05 Durability	Successful writes survive application restart; daily backup and a tested restore procedure before production.
NF06 Observability	Log request ID, route, status, duration and error category. Do not log tokens or full bug/comment content by default.
NF07 Consistency	Bug, activity and notification writes commit atomically. Simulated write failure must leave no partial result.
NF08 AI reliability	Test timeout, rate limit, invalid model output, prompt injection-like content and provider outage; core tracker remains usable.
NF09 Accessibility	Visible focus, labels, meaningful error text and status text independent of color.
NF10 Verification	Automate API, permission, lifecycle, retry, concurrency and AI guardrail tests; run UI smoke tests before release.

Architecture boundaries

A web frontend calls a REST API. The API owns authorization, validation, workflow and transactions. A relational datastore persists bugs and related records. An external identity provider authenticates users. An AI adapter communicates with the selected model/provider and returns schema-validated suggestions. The language, frontend framework, database engine, hosting and AI provider remain implementation decisions.

Deployment and data handling

Maintain separate development, test and production configuration; keep secrets out of source control; use versioned migrations; restrict runtime permissions; and test backup/rollback procedures. Confirm data retention, AI provider terms, and operational ownership before production.

10. Acceptance criteria and test cases

Tests should use Reporter A, Reporter B, Developer A, Triage Lead and an inactive account. Verify persisted state as well as API response and UI behavior.

Test	Given/action	Expected result and trace
TC01	Submit valid title, description and project.	201; unique reference; New status; authenticated reporter; history event. FR01.
TC02	Submit blank title, short description, missing project or unknown field.	422 field error; no partial records. FR02.
TC03	Test title/description min and max boundaries.	Only allowed boundary lengths pass. Section 6.
TC04	Request without authentication or as inactive user.	401; no bug or related records. P01.
TC05	Reporter A requests Reporter B's bug, comments or activity.	404; no data exposure or mutation. P03.
TC06	Reporter attempts agent-only status/assignment update.	403; state unchanged. P02/FR05.
TC07	Search and filter bugs across projects.	Only authorized matches; correct totals and ordering. FR03–04.
TC08	Triage lead assigns developer and moves Triaged → In Progress.	Success; one version increment; assignment/status history recorded. FR06–07.
TC09	Assign inactive user or Reporter as developer.	422; previous assignment and version preserved. FR06.
TC10	Attempt an invalid lifecycle transition.	409 INVALID_TRANSITION; no event or state change. Section 4.
TC11	Close a bug without verification evidence.	422; status remains unchanged. FR07.
TC12	Reopen a closed bug with a reason.	Reopened; prior fix and verification remain in history. P09.

Test	Given/action	Expected result and trace
TC13	Two users update the same version.	One succeeds; stale update returns 412; no lost update.
TC14	Retry create/comment with same idempotency key/body.	Original result returned; no duplicate bug/comment/notification.
TC15	Request AI triage and reject the suggestion.	Suggestion labeled; saved fields unchanged; rejection recorded. FR10/18.
TC16	AI returns malformed output or times out.	Output rejected or retryable error shown; core tracker remains usable. FR17/NF08.
TC17	Ask for similar bugs across unauthorized project.	No unauthorized bug identifiers or content returned. P05.
TC18	Generate test drafts and leave unapproved.	Drafts remain unapproved and are not marked as executed tests. FR12.
TC19	Enter HTML/script-like text in report/comment.	Stored/displayed safely as text; no script execution. NF01.
TC20	Change assignment and status in one request.	Correct audit events and one notification per applicable recipient.
TC21	Mark another user's notification as read.	404/forbidden; original read state unchanged.
TC22	Test narrow viewport and keyboard-only flow.	Forms, labels, focus, errors and primary actions remain usable. NF04/NF09.
TC23	Simulate database failure during event/notification write.	Entire transaction rolls back; retry can complete once. NF07.
TC24	Run agreed load test and restore backup.	Performance and recovery targets met with evidence. NF03/NF05.

Overall release acceptance

All source requirements and adopted proposed requirements must map to passing tests. The report-to-triage-to-fix-to-verification journey must pass end to end. No unresolved defect may permit unauthorized access, data loss, duplicate writes within the retry window, invalid lifecycle transitions or silent AI-driven changes. Record build, test environment, execution date and evidence. This specification defines tests; it does not claim an implementation has passed them.

11. Delivery plan and decisions

Implementation sequence

Establish identity integration, project membership, role enforcement and database schema. Verify access boundaries first.

Implement bug creation, validation, idempotency, scoped reads and list/detail screens. Verify required fields and New status.

Implement triage, assignment, priority/status lifecycle, comments, optimistic concurrency and immutable history.

Add AI suggestions behind an adapter; implement approval/rejection, similar-bug suggestions and test drafts with guardrail tests.

Implement notifications, dashboard, accessibility, operational controls and complete acceptance tests. Record release evidence.

Decisions requiring stakeholder review

Decision	Proposed baseline	Effect if changed
Identity	External identity provider; roles provisioned outside app	Changes auth integration and deployment.
Project scope	One organization with project boundaries	Multi-tenant support expands authorization and data model.

Decision	Proposed baseline	Effect if changed
AI provider	Replaceable model adapter; human approval required	Provider policy, latency, cost and privacy requirements change.
Workflow	New → Triaged → In Progress → Fixed → Ready for Verification → Closed; Reopened on failure	Additional states require transition and test changes.
AI automation	Suggestions only; no autonomous closure or code changes	More automation requires additional approval, audit and safety controls.
Notifications	In-app notifications first	Email/chat channels require delivery, retry and privacy contracts.
Attachments	Only if validated and implemented; otherwise excluded from UI	Requires storage, malware and access-control requirements.
Operations	Proposed performance/backup targets	Owner must confirm sizing, retention and recovery duties.

Specification change control

Store this specification with the implementation. For a requirement change, update the requirement, affected API/data contracts and tests before changing code. Record version, date, rationale and affected IDs. AI-generated code shall be reviewed against this baseline; a tool prompt does not override the specification.

Definition of ready

This draft is detailed enough to plan an implementation exercise. Before production, stakeholders must approve the scope and proposed rules, select the technology stack and AI provider, and assign owners for identity, data retention, hosting, monitoring and recovery.